

Virtual Memory

1. The Core Idea

Virtual memory lets a process's logical address space be **larger than physical RAM**, by keeping only the pages currently being used in RAM, and the rest on disk — brought in only when actually needed.

This is what makes **demand paging** possible: instead of loading a process's entire address space into memory before it runs, the OS loads pages **lazily, on demand**, the first time each one is actually accessed.

```
Logical address space (large, virtual)
|
Some pages in RAM (frames)   Some pages still on disk (swap space)
```

2. Page Faults

One-line definition: a page fault occurs when a process accesses a page that is **not currently in RAM** — the OS must fetch it from disk before execution can continue.

How it's detected

Every page table entry carries a **valid/invalid (present/absent) bit** (already introduced in Memory Management):

- **Valid = 1** → page is in memory → translation proceeds normally.
- **Valid = 0** → page fault → **trap to the OS**.

Steps in page fault handling

1. TRAP to OS (hardware detects invalid bit, transfers control to kernel)
2. Locate the required page (find it in secondary storage)
3. Find a free frame — if none is free, select a victim page to replace (page replacement algorithm)
4. Load the required page into that frame, update the page table
5. Restart the instruction that caused the fault

Why "restart" and not "resume"? The instruction that triggered the fault hadn't completed — its operand wasn't available yet. Once the page is loaded, the *entire instruction* is re-executed from the start (not resumed mid-way), since the CPU generally can't resume a partially-executed instruction cleanly.

Effective Access Time (EAT) with page faults

$$\text{EAT} = (1 - p) \times \text{memory_access_time} + p \times \text{page_fault_service_time}$$

where p = probability of a page fault.

Why this matters: page fault service time is **dominated by disk I/O** — often on the order of *milliseconds*, compared to memory access times measured in *nanoseconds*. This means even a very small page fault probability can dramatically inflate EAT.

Worked example: memory access time = 100ns, page fault service time = 8ms = 8,000,000ns.

- If $p = 0.001$ (0.1% of accesses fault):

$\text{EAT} = (0.999)(100) + (0.001)(8,000,000) \approx 99.9 + 8000 = 8099.9\text{ns}$ — roughly **80x slower** than memory access alone, from a tiny fault probability. This is precisely *why* minimizing page faults (via good replacement algorithms and adequate frame allocation) matters so much.

3. Page Replacement Algorithms

When does replacement matter? When a page fault occurs and **no free frame is available** — the OS must pick an existing page to evict (the "victim") to make room.

Goal: minimize the total number of page faults over the life of the process.

3.1 FIFO (First-In, First-Out)

Replace the page that has been in memory the **longest**, regardless of how recently or often it's been used.

Example (frames = 3, reference string: **1 5 1 0 2 1 5 1 2 7 0 1**):

Ref	1	5	1	0	2	1	5	1	2	7	0	1
Frame set	{1}	{1,5}	{1,5}	{1,5,0}	{5,0,2}	{0,2,1}	{2,1,5}	{2,1,5}	{2,1,5}	{1,5,7}	{5,7,0}	{7,0,1}
Fault ?	F	F		F	F (evict 1)	F (evict 5)	F (evict 0)			F (evict 2)	F (evict 1)	F (evict 5)

Eviction always removes whichever page has sat in memory **longest**, regardless of recent use — e.g., at the 5th reference, frame set **{1,5,0}** is full, so page **1** (the oldest arrival) is evicted to admit **2**, even though **1** is about to be needed again two references later. **Total: 9 faults out of 12 references.**

- **Advantage:** simple to implement (just a queue).

- **Disadvantage:** doesn't consider how *often* or how *recently* a page is actually used — can evict a page that's about to be reused, right after evicting it. This is also the algorithm subject to **Belady's Anomaly** (below).

3.2 Optimal (OPT / MIN)

Replace the page that **will not be used for the longest time in the future**.

- **Advantage:** provably the **minimum possible** number of page faults for any reference string — this is the theoretical benchmark every other algorithm is measured against.
- **Disadvantage: impossible to implement in practice** — it requires knowing the future reference string in advance, which the OS cannot know. Its value is purely as an upper bound for comparison.

3.3 LRU (Least Recently Used)

Replace the page that has **not been used for the longest time in the past** — using past behavior as a proxy for future behavior (the opposite direction of Optimal, which needs the *actual* future).

Example (frames = 3, reference string: 7 0 1 2 0 3 0 4 2 3):

Ref	7	0	1	2	0	3	0	4	2	3
Frame set	{7}	{7,0}	{7,0,1}	{0,1,2}	{0,1,2}	{0,2,3}	{0,2,3}	{0,3,4}	{0,4,2}	{4,2,3}
Fault?	F	F	F	F (evict 7)		F (evict 1)		F (evict 2)	F (evict 3)	F (evict 0)

Two hits, at the two repeated 0 references (positions 5 and 7) — each time, 0 had been used recently enough to survive as the least-recently-used candidate. **Total: 8 faults out of 10 references** — better than FIFO would do on a comparably adversarial string, because eviction tracks actual recency of use rather than arrival order.

- **Advantage:** a strong practical approximation of OPT — because of **locality of reference** (recently used pages tend to be used again soon), the recent past is usually a good predictor of the near future.
- **Disadvantage:** implementation cost — precisely tracking "how recently was each page used" requires either a counter/timestamp updated on every access, or a linked-list structure reordered on every access; both add real overhead. (Approximations like the "second-chance"/clock algorithm exist specifically to reduce this cost — worth mentioning if asked for more depth.)

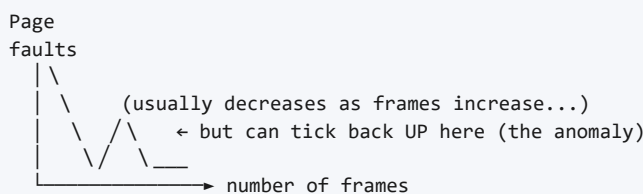
3.4 LFU (Least Frequently Used) / MFU (Most Frequently Used)

- **LFU** — replace the page with the **fewest total accesses**. Rationale: a rarely-used page is less likely to be needed again.

- **Problem:** a page that was heavily used early on (e.g., during process startup) but is no longer needed can accumulate a high count and **never get evicted**, even though it's now genuinely cold — the count reflects history, not current relevance.
- **MFU** — replace the page with the **most** accesses, on the reasoning that a page just brought in and used heavily has *already* served its purpose and is less likely needed again soon. (Counter-intuitive, and rarely used in practice — mentioned mainly for completeness/contrast with LFU.)

3.5 Belady's Anomaly

Definition: for *some* page replacement algorithms (most notably **FIFO**), increasing the number of available frames can — counter-intuitively — **increase** the number of page faults, rather than decrease it.



Why this happens: FIFO's eviction choice depends only on arrival order, not on usage — adding more frames changes *which* pages happen to still be resident at each step in a way that isn't guaranteed to be monotonically beneficial.

Key algorithmic fact: algorithms in the "**stack algorithm**" family (LRU and Optimal both qualify) are **provably immune** to Belady's Anomaly — for these, adding frames can never increase page faults. FIFO is *not* a stack algorithm, which is exactly why it's vulnerable.

Interview soundbite: "Belady's Anomaly is the counter-intuitive result that FIFO page replacement can produce *more* page faults with *more* frames. It happens because FIFO's eviction order depends only on arrival time, not on usage — LRU and Optimal don't suffer from this because they belong to the 'stack algorithm' class, where a frame-set at size n is always a subset of the frame-set at size $n+1$, which structurally guarantees faults can only decrease or stay the same as frames increase."

Summary Comparison

Algorithm	Basis	Optimality	Practical?
FIFO	Arrival order	Weak; suffers Belady's Anomaly	Yes, simple
Optimal	True future knowledge	Provably best possible	No (needs the future)
LRU	Recent past usage	Close approximation to Optimal	Yes, with some overhead
LFU	Total access frequency	Can trap stale-but-once-popular pages	Rarely used alone
MFU	Total access frequency (inverse)	Counter-intuitive, rarely optimal	Rarely used

4. Global vs Local Replacement

When a page fault occurs and a victim must be chosen, **where can that victim come from?**

	Global Replacement	Local Replacement
Victim can come from	Any process in memory	Only the faulting process's own frames
Effect	One process's fault can steal a frame from a completely unrelated process	A process's performance depends only on itself
Advantage	Better <i>overall</i> system-wide memory utilization; can reduce total system-wide page faults	Isolation — predictable, stable per-process performance
Disadvantage	Unpredictable — one process can degrade another's performance; can itself contribute to thrashing	May be less efficient overall (a process with unused frames won't share them with one that's struggling)

The key trade-off: Global replacement optimizes for *system-wide* throughput at the cost of *process-level* predictability/isolation. Local replacement optimizes for *isolation and stability* at the cost of potentially wasting frames that sit idle in one process while another starves.

5. Frame Allocation

One-line definition: frame allocation decides how the OS distributes a limited number of physical frames among competing processes.

5.1 Equal Allocation

Split frames evenly across all processes.

Example: 90 frames, 3 processes → 30 frames each.

- ✓ Simple, fair by count.
- ✗ Ignores actual process size/need — a tiny process gets the same allocation as a huge one, wasting frames on the small process while starving the large one.

5.2 Proportional Allocation

Allocate frames **proportional to process size**.

Frames for $P_i = (\text{Size of } P_i / \text{Total Size}) \times \text{Total Frames}$

Worked example: total physical memory = 1200KB, page/frame size = 4KB → total frames = $1200/4 = 300$ frames. Processes with needed-frame counts 40, 60, 100, 20, 80, 100 (summing to 400):

```
P0: (40/400) × 300 = 30 frames
P1: (60/400) × 300 = 45 frames
P2: (100/400) × 300 = 75 frames
P3: (20/400) × 300 = 15 frames
P4: (80/400) × 300 = 60 frames
P5: (100/400) × 300 = 75 frames
```

- ✓ Matches allocation to actual demand — more efficient than equal allocation.
- ✗ Slightly more complex to compute and maintain as process sizes/needs change dynamically.

5.3 Priority Allocation

Allocate more frames to higher-priority processes, regardless of size — an important process gets more room to run efficiently.

- ✗ Risk: low-priority processes can be starved of frames.

Minimum frames — a hard constraint

Every process needs **at least enough frames to execute a single instruction** — if an instruction (with its operands) spans, say, 3 pages, the process needs a **minimum of 3 frames** just to make any progress at all, regardless of which allocation strategy is chosen. This is a hardware/architecture-driven floor, not a policy choice.

Allocation is linked to replacement scope

- **Global allocation** ↔ frames can be dynamically taken from any process (see §4).
- **Local/fixed allocation** ↔ each process is locked to its allotted frame count; replacement happens only within it.
- **Dynamic allocation** — the OS can adjust a process's frame count *during* execution — e.g., increase frames if a process is faulting heavily, decrease if it's comfortably under-using its allocation. This is exactly the bridge to the Working Set model below.

6. The Working Set Model

One-line definition: the working set is the set of pages a process is **actively using** at a given point in time — the subset it genuinely needs resident in memory to run without excessive faulting.

Why this exists

A process almost never touches its *entire* address space at once — thanks to **locality of reference**, it tends to stay concentrated in a small, slowly-shifting region for stretches of time (a loop body, a function's local data, etc.). Frame allocation decisions are much better informed by "how big is this region right now" than by a static per-process frame count.

Formal definition

Working set = the set of distinct pages referenced in the most recent Δ (delta) time units (Δ can be measured in real time, or in a number of most-recent memory references).

Worked example: reference string `1 2 3 2 1 4 5 4`, with $\Delta = 4$:

- Looking at the most recent 4 references at each point, the working set shifts as execution proceeds — e.g., at one point it might be `{1, 2, 3}`, and a bit later `{1, 2, 3, 4}` as the process's locality shifts to include page 4.

Working Set Size (WSS) and total demand

- **WSS** = number of *distinct* pages in the current working set.
- **Total Demand** (with multiple processes) = `WSS1 + WSS2 + ... + WSSn`.

The key operating rule:

- If `Total Demand ≤ Available Frames` → the system can comfortably keep every process's working set resident → efficient execution.

- If **Total Demand > Available Frames** → there aren't enough frames to keep everyone's actively-needed pages loaded simultaneously → **thrashing** (below).

How the OS uses this model

1. Estimate each process's working set (its WSS) — typically approximated, not tracked with perfect precision.
 2. Allocate frames aligned with each process's actual working set need.
 3. If total demand still exceeds available frames even after reasonable allocation, **suspend some processes entirely** (moving them out of the "actively running" set, e.g., into the 7-state model's suspended states from Process Management) rather than let everyone starve simultaneously.
-  Directly prevents thrashing; leads to efficient, need-based memory allocation.
 -  Genuinely hard to track precisely in real time (the working set shifts continuously), and computing/maintaining it has real overhead.

7. Thrashing

One-line definition: thrashing is the state where the system spends **more time paging (swapping pages in/out) than actually executing processes** — throughput collapses even though the CPU and disk both look "busy."

The vicious cycle

```
Too many processes admitted (over-ambitious degree of multiprogramming)
|
Each process gets too few frames (below its actual working-set need)
|
Frequent page faults for everyone
|
CPU utilization appears to drop → OS's naive response: "add MORE processes to keep CPU busy"
|
Even less memory per process → even MORE page faults
|
(repeats, spiraling downward)
```

The counter-intuitive trap: if the OS misreads low CPU utilization as "not enough processes competing for the CPU" and responds by increasing the degree of multiprogramming, it makes thrashing **worse**, not better — because the real cause was *too little memory per process*, not too few processes.

Why the working set model (and good frame allocation generally) is the fix

By recognizing thrashing as a symptom of **Total Demand > Available Frames**, the correct response is the *opposite* of the naive one: **reduce** the degree of multiprogramming (suspend some processes) so

the remaining ones can each get enough frames to hold their actual working sets, restoring efficient execution for those that keep running.

Interview soundbite: "Thrashing happens when there isn't enough memory to hold the working sets of all running processes, causing constant page faulting that starves actual computation. The dangerous part is that it can masquerade as low CPU utilization, tempting a naive scheduler to add *more* processes — which only deepens the problem. The correct fix is to reduce the degree of multiprogramming, giving the remaining processes enough frames to actually run efficiently."

Interview Questions With Answers

Q1. What is a page fault, and what are the steps the OS takes to handle one?

Answer: A page fault occurs when a process accesses a page marked invalid/not-present in its page table — meaning the page it needs isn't currently in RAM. The OS traps into kernel mode, locates the page on secondary storage, finds a free frame (running a page replacement algorithm to evict a victim if none is free), loads the required page into that frame and updates the page table, and then restarts the instruction that originally faulted from the beginning.

Q2. Why does even a very small page fault probability significantly impact Effective Access Time?

Answer: Because page fault service time (dominated by disk I/O) is many orders of magnitude slower than a normal memory access — milliseconds vs. nanoseconds. In $EAT = (1-p) \times \text{memory_access} + p \times \text{fault_service_time}$, even a tiny p gets multiplied by a value that's roughly a million times larger than the memory access term, so it can dominate the overall EAT despite being a "rare" event.

Q3. Why is the Optimal page replacement algorithm not actually usable in real systems?

Answer: Optimal requires knowing, at the moment of replacement, exactly which page will be used furthest in the future — which requires knowing the process's entire future reference pattern in advance. No real OS can know the future, so Optimal exists purely as a theoretical best-case benchmark to measure other algorithms against, not as something that can actually run.

Q4. Why does LRU generally perform well in practice despite not being optimal?

Answer: LRU relies on locality of reference — the empirical observation that programs tend to keep reusing a small, slowly-shifting set of pages over any short window. Because of this, "least recently used" tends to correlate strongly with "least likely to be used again soon," making LRU's guess about the future (based on the past) usually quite accurate — close to what Optimal would have chosen, without needing to actually know the future.

Q5. What is the core problem with LFU?

Answer: LFU replaces based on total access count, which reflects a page's *entire history*, not its current relevance. A page that was accessed heavily early in a process's life (e.g., during startup/initialization) can accumulate a high count and then never get evicted even after it becomes genuinely unused, because its historical count keeps it looking "important" relative to newer pages that just haven't had time to accumulate as many accesses yet.

Q6. What is Belady's Anomaly, and which algorithms are immune to it?

Answer: Belady's Anomaly is the counter-intuitive scenario (most notably in FIFO) where increasing the number of available frames actually *increases* the number of page faults instead of decreasing them. Stack algorithms — including LRU and Optimal — are provably immune to it, because their frame-set at any given size is always a subset of the frame-set at the next larger size, which guarantees faults can never increase as frames increase. FIFO is not a stack algorithm, which is exactly why it's vulnerable.

Q7. What's the difference between global and local page replacement, and what's the key trade-off?

Answer: Global replacement allows a victim page to be selected from any process currently in memory, while local replacement restricts victim selection to only the faulting process's own allocated frames. The trade-off: global replacement can improve overall system throughput (since frames flow to wherever they're most needed) but sacrifices predictability and isolation (one process's heavy faulting can steal frames from, and degrade, an unrelated process). Local replacement gives stable, isolated per-process performance at the potential cost of overall efficiency (a process with spare frames won't share them with one that's struggling).

Q8. Compare equal, proportional, and priority frame allocation.

Answer: Equal allocation splits frames identically across all processes regardless of size or need — simple and fair by count, but wasteful (small processes over-provisioned, large ones under-provisioned). Proportional allocation gives each process a frame count proportional to its size relative to total size — more efficient and demand-matched, at the cost of slightly more computation to maintain. Priority allocation gives more frames to higher-priority processes regardless of size, which improves performance for important processes but risks starving low-priority ones.

Q9. Why must every process be guaranteed a minimum number of frames?

Answer: Because an instruction (with its operands) may itself span multiple pages — if an instruction requires 3 pages to execute at all (e.g., the instruction spans a page boundary, plus it references an operand on another page), the process needs at least 3 frames just to execute that single instruction. This minimum is dictated by the hardware/instruction-set architecture, not by any allocation policy choice.

Q10. What is the working set of a process, and how does it relate to thrashing?

Answer: The working set is the set of distinct pages a process has referenced within the most recent Δ time units (or Δ references) — essentially, the pages it's actively relying on right now due to locality of reference. Thrashing is directly tied to this: if the sum of every running process's working set size (total demand) exceeds the number of available frames, there simply isn't enough memory to keep everyone's actively-needed pages resident simultaneously, forcing constant page faulting — which is exactly what thrashing is.

Q11. Explain the "vicious cycle" of thrashing, and why a naive scheduler response makes it worse.

Answer: When too many processes are admitted relative to available memory, each gets too few frames to hold its actual working set, causing frequent page faults. This heavy faulting activity makes the CPU look underutilized (it's busy waiting on disk I/O, not doing computation), and a naive scheduler seeing "low CPU utilization" might respond by admitting *even more* processes to try to keep the CPU busier. But this only shrinks the average frames-per-process further, causing even more faulting — deepening the thrashing rather than fixing it. The correct response is the opposite: reduce the degree of multiprogramming so remaining processes get enough frames to actually make progress.

Q12. Scenario: A system is experiencing very low CPU utilization, and the OS's default policy is to increase the degree of multiprogramming whenever it sees this. What could go wrong, and how would you diagnose the real cause?

Answer: If the low CPU utilization is actually caused by thrashing (processes spending most of their time waiting on page faults rather than genuinely idle due to too few processes), blindly increasing the degree of multiprogramming will make things dramatically worse — more processes competing for the same limited frames means even smaller allocations per process, even more faulting, and utilization drops further, potentially triggering the scheduler to add still more processes in a runaway spiral. To diagnose correctly, you'd check the page fault rate alongside CPU utilization: low utilization *with* a high fault rate points to thrashing (the fix is to *reduce* multiprogramming, e.g., via the working set model, and give remaining processes more frames), whereas low utilization *with* a low fault rate suggests genuinely too few processes are ready to run (where increasing multiprogramming would actually help).